

普及组初赛模拟题（九）

一、单项选择题（共15题，每题2分，共计30分；每题有且仅有一个正确选项）

1. 假设 $A = TRUE$, $B = FALSE$, $C = TRUE$, $D = TRUE$, 逻辑运算表达式 $A \wedge B \vee C \wedge D$ 的值是:
 - A. $TRUE$
 - B. 0
 - C. 1
 - D. $FALSE$
2. 设 $a[1] = 1, a[2] = 2, a[3] = 3, a[4] = 4, a[5] = 5, a[6] = 6$, 且 $i = 1, j = 2, k = 3, m = 4$, 以下选项中变量值等于 3 的是:
 - A. $a[i * j]$
 - B. $a[a[j + k - 2]]$
 - C. $a[a[k - i] + 3]$
 - D. $a[m / j]$
3. 八进制数 15264 转换成的二进制数是 () 。
 - A. 001101010110100
 - B. 101101011010101
 - C. 001011000110100
 - D. 001101010100110
4. 一个二叉搜索树前序遍历的结果为 {7、2、1、5、13、9}, 这棵树的根的左子树有多少个节点? () 。
 - A. 2
 - B. 5
 - C. 4
 - D. 3
5. 小明一次可以跨上一阶台阶或两阶台阶。他从第一阶台阶出发, 有多少种方案恰好到达第 10 阶台阶? ()
 - A. 53
 - B. 55
 - C. 54
 - D. 57
6. 所有满足除以 3 的余数为 2, 除以 7 的余数为 1 中, 最小的数是 8, 第五小的数是 () 。

- A. 40
- B. 50
- C. 71
- D. 92

7. 2021 年 7 月 1 日是星期四，我们热烈庆贺了中国共产党成立一百周年，完成了两个一百年中的第一个。我们将在 2049 年 10 月 1 日欢庆建国一百年，这一天是星期几？（ ）。

- A. 星期四
- B. 星期五
- C. 星期六
- D. 星期日

8. 一棵 6 节点二叉树的中序遍历为 DBAGECF，先序遍历为 ABDCEGF，后序遍历为（ ）。

- A. DGBEFAC
- B. GBEACFD
- C. DBGEFCA
- D. ABCDEFG

9. G 是一张有 n 个点 m 条边的连通图，必须删去（ ）条边才能将其变成一棵 n 节点的树

- A. 1
- B. $m - n - 1$
- C. $m + n - 1$
- D. $m - n + 1$

10. 元素 R_1 、 R_2 、 R_3 、 R_4 、 R_5 入栈的顺序为 R_1 、 R_2 、 R_3 、 R_4 、 R_5 。如果第一个出栈的是 R_3 ，那么第 5 个出栈的不可能是（ ）。

- A. R_1
- B. R_2
- C. R_4
- D. R_5

11. 前缀表达式 “ $+3 * 2 + 5 \ 12$ ” 的值是（ ）。

- A. 23
- B. 25
- C. 37
- D. 65

12. 设有 100 个顶点，利用二分法查找时，最大比较次数是（ ）。

- A. 25
- B. 10
- C. 7
- D. 50

13. 十进制小数 13.375 对应的二进制数是（ ）：

- A. 1101.011

- B. 1011.011
- C. 1101.101
- D. 1010.01

14. $1TB$ 表示的字节数是 () 。

- A. 2^{10}
- B. 2^{20}
- C. 2^{30}
- D. 2^{40}

15. 有 18 个外观完全相同的零件，其中有且仅有一个是质量略轻的次品零件。有一个天平，但是没有砝码，可以用于比较左右两个盘中物体的质量。最坏情况下，最少需要多少次称量可以找到这个次品零件？ () 。

- A. 3
- B. 4
- C. 6
- D. 9

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 T，错误填 F；除特殊说明外， 判断题 1.5 分，选择题 3 分，共计 40 分）

注意：判断题正确填 T，错误填 F。

阅读下面程序，完成第 16 ~ 21 题。

```

1 | #include <iostream>
2 |
3 | #include <cstring>
4 |
5 | using namespace std;
6 | int main() {
7 |     int t[256];
8 |     char s[15];
9 |     int i;
10 |    scanf("%s", s);
11 |    for (i = 0; i < 256; i++) t[i] = 0;
12 |    for (i = 0; i < strlen(s); i++) t[s[i]]++;
13 |    for (i = 0; i < strlen(s); i++)
14 |        if (t[s[i]] == 1) {
15 |            cout << s[i] << endl;
16 |            return 0;
17 |        }
18 |    cout << "no" << endl;
19 |    return 0;
20 | }

```

16. 程序会将读入的字符串中出现次数等于 1 的字符依次输出。[]

17. 将程序中的

```

1 | for (i = 0; i < 256; i++) t[i] = 0;

```

删掉可能会影响程序的正确性。[]

18. 当程序读入字符串的内容是 "xyzxyw" 时，程序的输出结果是 w 。[]

19. 对于一组输入数据 "abc?ac" ， 如果希望程序的输出结果为 b ， 则 ? 处应替换为 b 。[]

20. 若输入的字符串中 a 到 g 这 7 种字符均至少出现一次， 如果希望输出结果为 no ， 则输入的字符串长度至少为()。

- A. 1
- B. 7
- C. 14
- D. 8

21. 这个程序最多正确输入并处理长度为()的字符串 s 。

- A. 256
- B. 15
- C. 14
- D. 11

阅读下面程序，完成第 22 ~ 27 题。

```

1  #include <iostream>
2
3  using namespace std;
4  const int maxn = 105;
5  int n, a[maxn], b[maxn];
6  int main() {
7      cin >> n;
8      int x;
9      for (int i = 1; i <= n; i++) {
10         cin >> x;
11         a[i] = b[i] = x;
12     }
13     for (int i = 1; i <= n; i++)
14         for (int j = i + 1; j <= n; j++) {
15             if (a[i] > a[j]) swap(a[i], a[j]);
16             if (b[i] < b[j]) swap(b[i], b[j]);
17         }
18     for (int i = 1; i <= n; i++) cout << a[i] << " ";
19     cout << "\n";
20     for (int i = 1; i <= n; i++) cout << b[i] << " ";
21     cout << "\n";
22     return 0;
23 }

```

22. 若输入的 $X[1], X[2], \dots, X[N]$ 中有相同的数,程序会陷入死循环。[]
23. 当且仅当输入的 $X[1], X[2], \dots, X[N]$ 全部相同时输出的两行结果相同。[]
24. 该算法的原理是基数排序。[]
25. (4分) 若输入的 $X[1], X[2], \dots, X[N]$ 互不相同, 则下列说法正确的是()
- A.输出的两行结果相同
 - B.将输出的第一行结果整体翻转后, 将与第二行相同
 - C.将输出的第一行结果的第一项与最后一项交换后, 将与第二行相同
 - D.以上说法都不正确
26. (4分) 下列说法不正确的是()
- A.输出的第一行即为将 $X[1], X[2], \dots, X[N]$ 从小到大排序后得到的结果
 - B.输出的第二行即为将 $X[1], X[2], \dots, X[N]$ 从大到小排序后得到的结果
 - C.若将 $a[i] > a[j]$ 改为 $a[i] \geq a[j]$, 则程序输出无变化
 - D.不存在时间复杂度更优的能与本程序达到相同目的算法
27. 该程序的时间复杂度为()
- A. $O(n \log n)$
 - B. $O(n)$
 - C. $O(n^2)$
 - D. $O(n\sqrt{n})$

阅读下面程序, 完成第 28 ~ 33 题。

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  int a[6];
4  int change(int a) {
5      a++;
6  }
7  int change1(int & a) {
8      a++;
9  }
10 int main() {
11     int c = 1;
12     for (int i = 1; i <= 5; i++) a[i] = i * 3;
13     int * b = & a[1];
14     change( * b);
15     cout << * b << endl;
16     cout << a[1] << endl;
17     * b++;
18     cout << * b << endl;
19     cout << a[1] << endl;
20     change1( * b);
21     cout << * b << endl;
22     cout << a[1] << endl;
23     * b = c;
24     change(c);
25     cout << * b << endl;
26     cout << c << endl;
27     change1(c);
28     cout << * b << endl;
29     cout << c << endl;
30     return 0;
31 }

```

28. 将 `int c = 1;` 中 `int` 换为 `long long` 后程序依然能通过编译。[]

29. `change` 与 `change1` 两个函数等价。[]

30. 将 `* b = c;` 换为 `b = &c;` 输出值不变。[]

31. 将 `int * b = & a[1];` 换为 `int *b=a+1;` 输出值不变。[]

32. 输出结果的最大值是。()

- A. 6
- B. 4
- C. 7
- D. 5

33. 输出结果的乘积是。()

- A. 13608
- B. 11520
- C. 5760

三、完善程序（单选题，每小题3分，共计30分）

阅读下面题目，完成第 34 ~ 38 题。

Takahashi目前处在一个二维平面上，初始状态下Takahashi在点 $(0, 0)$ 。

Takahashi会进行 n 次如下的移动，每次移动Takahashi有三种选择：

1. 从 (x, y) 移动到 $(x + A, y + B)$ 。
2. 从 (x, y) 移动到 $(x + C, y + D)$ 。
3. 从 (x, y) 移动到 $(x + E, y + F)$ 。

现在在 m 个点 $(X_1, Y_1), \dots, (X_m, Y_m)$ 有障碍物，Takahashi不能移动到含有障碍物的点上。

在经历了 n 次的移动后会得到多少条路径，答案对于 998244353 进行取模。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4  const int N = 330;
5  const int M = 100010;
6  const ll Mod = 998244353;
7  ll x[M], y[M];
8  set<pair<ll, ll >> p;
9  ll dp[N][N][N];
10 int main() {
11     int n, m;
12     ll a, b, c, d, e, f;
13     int i, j, k;
14     ll ans;
15     scanf("%d%d", &n, &m);
16     scanf("%lld%lld%lld%lld%lld%lld", &a, &b, &c, &d, &e, &f);
17     for (i = 1; i <= m; i++) {
18         scanf("%lld%lld", &x[i], &y[i]);
19     }
20     for (i = 1; i <= m; i++) {
21         p.insert({x[i], y[i]});
22     }
23     ans = 0;
24     dp[0][0][0] = ①;
25     for (i = 0; i <= n; i++) {
26         for (j = 0; i + j <= n; j++) {
27             for (k = 0; ②; k++) {
28                 if (i + j + k == n) {
29                     ans = (ans + dp[i][j][k]) % Mod;
30                     continue;
31                 }
32                 if (③) {
33                     dp[i + 1][j][k] = (dp[i + 1][j][k] + dp[i][j][k]) % Mod;
34                 }
35                 if (!p.count({i * a + (j + 1) * c + k * e, i * b + (j + 1) * d + k
36                     dp[i][j + 1][k] = (dp[i][j + 1][k] + dp[i][j][k]) % Mod;
37                 }
38                 if (!p.count({i * a + j * c + (k + 1) * e, i * b + j * d + (k + 1)
39                     }
40             }
41         }
42     }
43     printf("④\n", ans);
44     return 0;
45 }

```

34. ①处应填()

- A. -1
- B. 0

- C. 1
- D. 1e18

35. ②处应填()

- A. $i+j+k \leq n$
- B. $i+j+k < n$
- C. $j+k \leq n$
- D. $j+k < n$

36. ③处应填()

- A. `!p.count({i * a + j * c + k * e, i * b + j * d + k * f})`
- B. `!p.count({(i + 1) * a + j * c + k * e, (i + 1) * b + j * d + k * f})`
- C. `!p.count({(i + 1) * a + j * c + k * e, i * b + j * d + k * f})`
- D. `!p.count({i * a + j * c + k * e, (i + 1) * b + j * d + k * f})`

37. ④处应填()

- A. `%lld`
- B. `%d`
- C. `lld`
- D. `d`

38. ⑤处应填()

- A. `dp[i][j][k + 1] = (dp[i][j][k + 1] + dp[i][j][k]) % Mod;`
- B. `dp[i][j][k + 1] = dp[i][j][k] % Mod;`
- C. `dp[i][j][k + 1] = (dp[i][j][k + 1] * dp[i][j][k]) % Mod;`
- D. `dp[i][j][k + 1] = (dp[i][j][k + 1] - dp[i][j][k] + Mod) % Mod;`

阅读下面题目，完成第 39 ~ 43 题。

给定一个字符串S，有q组询问，每次给定一个字符串T，求字符串T是否是S中的一个子序列。
数据保证，所有字符串仅包含小写字母。

```

1  #include <iostream>
2  using namespace std;
3  const int BASE = 26;
4  string S, T;
5  int q, Pos[100005][BASE];
6  int main() {
7      cin >> S;
8      int n = S.size();
9      for (int i = 0; i < BASE; i++) {
10         ① = -1;
11     }
12     -1;
13     for (int i = n - 1; i >= 0; i--) {
14         for (int j = 0; j < BASE; j++) {
15             ②;
16         }
17         ③;
18     }
19     cin >> q;
20     while (q--) {
21         cin >> T;
22         int len = T.size(), now = 0;
23         for (int i = 0; ④; i++) {
24             now = ⑤;
25         }
26         if (now != -1) {
27             cout << "YES\n";
28         }
29         else cout << "NO\n";
30     }
31     return 0;
32 }

```

39. ①处应填 ()

- A. Pos[i][n - 1]
- B. Pos[i][n]
- C. Pos[n - 1][i]
- D. Pos[n][i]

40. ②处应填 ()

- A. Pos[i][j] = Pos[i - 1][j]
- B. Pos[i][j] = Pos[i + 1][j]
- C. Pos[i][j] = Pos[i][j - 1]
- D. Pos[i][j] = Pos[i][j + 1]

41. ③处应填 ()

- A. Pos[i][S[i] - 'a'] = i
- B. Pos[i][S[i] - 'A'] = i

- C. `Pos[i][S[i]] = i`
- D. `Pos[i][i] = S[i]`

42. ④处应填 ()

- A. `i < len`
- B. `now != -1`
- C. `i < len && now != n`
- D. `i < len && now != -1`

43. ⑤处应填 ()

- A. `Pos[now][T[i] - 'a']`
- B. `Pos[now][S[i] - 'a']`
- C. `Pos[now + 1][T[i] - 'a']`
- D. `Pos[now + 1][S[i] - 'a']`